



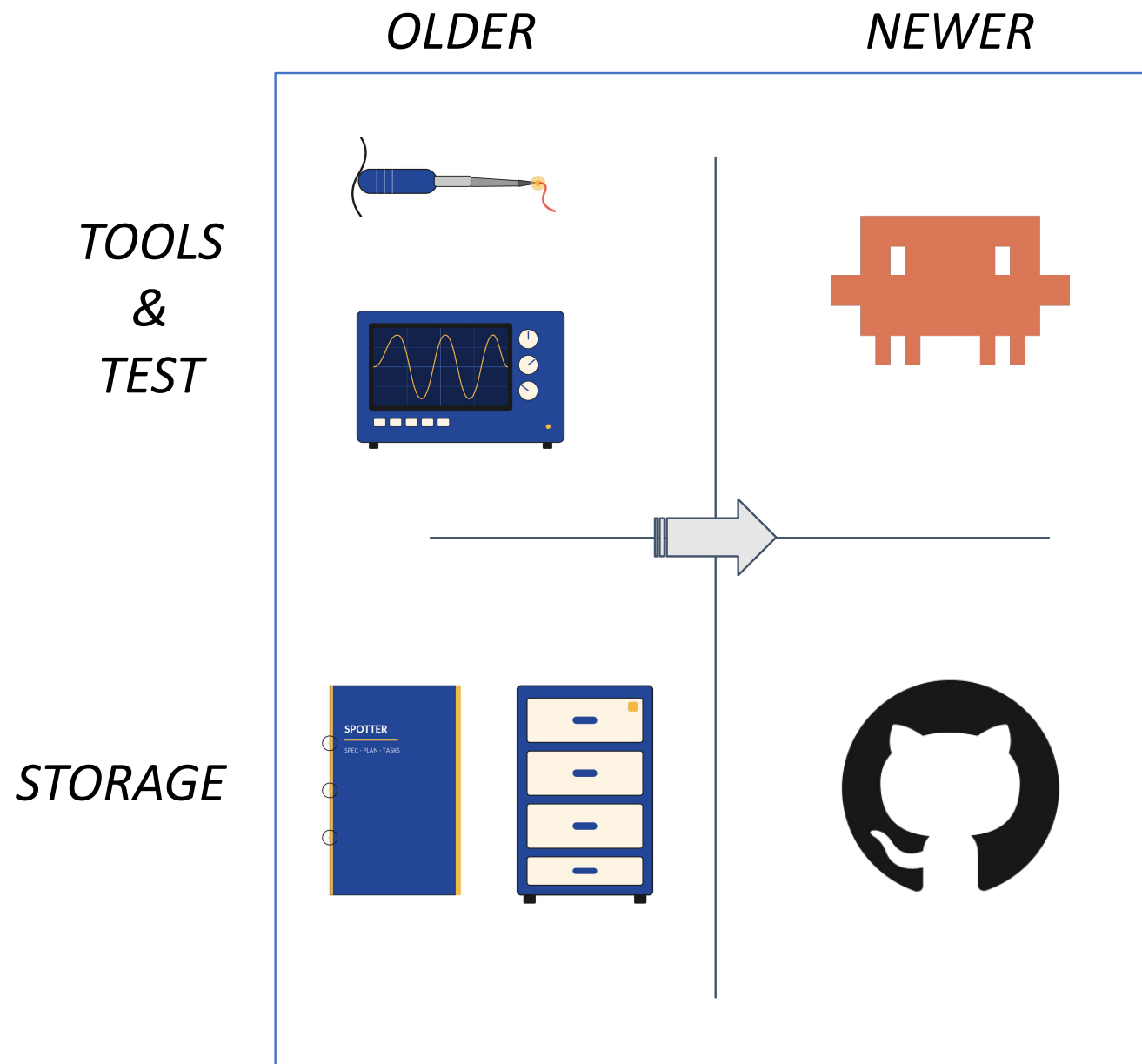
The Spec Is the New Schematic

Reproducible Station Software with Spec-Driven Development (SDD)

John Carter, N6YU — RSGB Convention 2026



■ Software Is Replacing Solder



- Today's shack: seems like as much software as hardware
- Adding a feature: hard in open source, harder in commercial gear, hardest in closed source
- New Tool Stack - OLD + Software Agents, GitHub
- GitHub is the new file cabinet

Using AI for coding has big drawbacks - **Not**

- Repeatable
- Structured

Key Terms

Agent — AI that edits files, runs code

Claude Code — AI agent in your terminal

CLAUDE.md — Notes Claude reads at startup

Commit — A saved file snapshot with a note

Context window — The AI's working memory

GitHub — Storage website; where files are tracked

Markdown (.md) — Formatted plain text

Masterplan — **Detailed Specs, Tasks... for your app**

Masterplan Generator — **Helper to customize MP**

Push — Upload your files to GitHub

Repo — Project folder plus its history

SDD — AI-updated plan of record

Session — One continuous AI conversation

Slash command — Shortcuts /usage, /status, etc.

Specs — **Your app's features and behaviors**

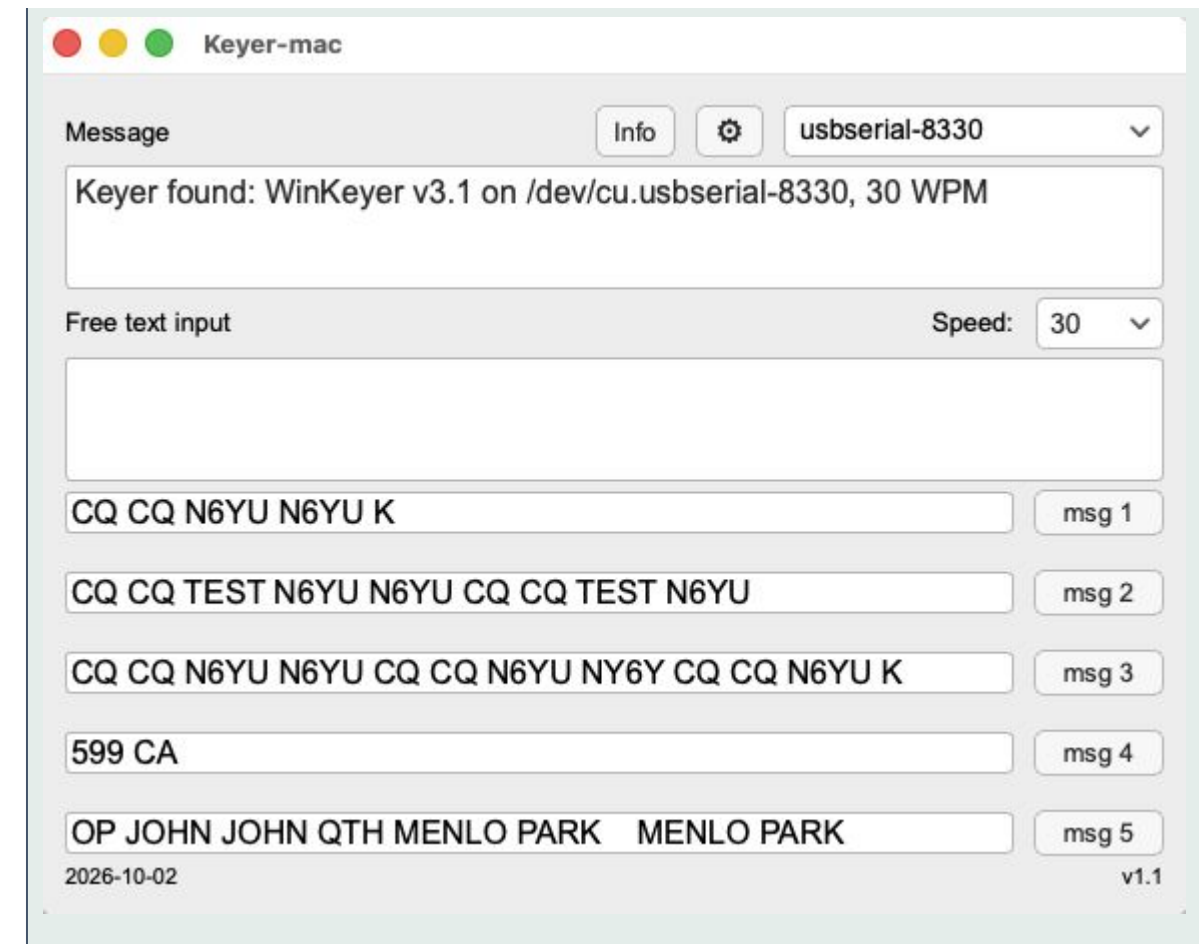
Terminal — Text-only app, no mouse

Token — Word-piece used to meter AI use

■ Possible SDD Projects for Radio

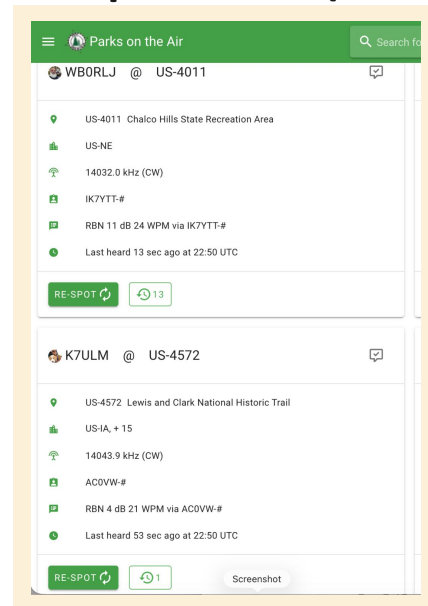
- Platform changes (Win<>Mac<>Linux<>Mobile)
- GitHub building assistance
- Glue & Controllers
 - Auto-Tuner, Shack Control
- Arduino/Raspberry Pi Projects
- Instrumentation & Automation
 - SDR, Filtering, Temp, Voltage, Current, SWR
- Logging, Tracking Awards, Spotting, Alerts, Band Activity, Antenna Testing
- Shack Control: Amps, Switching, Tuners
- Remote Operation

*SDD Example of Keyer built from K6GTE
Open source Linux -> Mac*

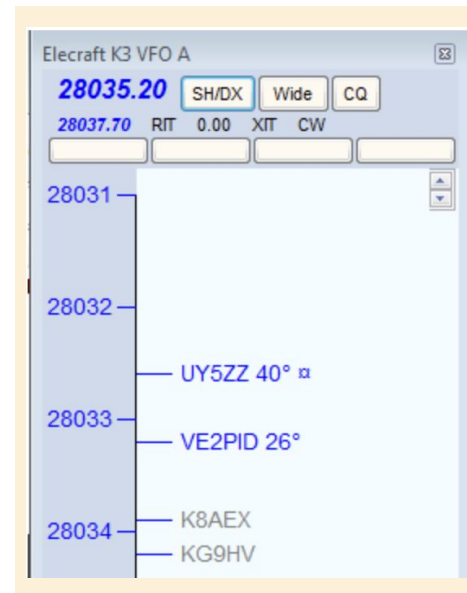


■ Why Did I Bother? A Poor Man's Panadapter

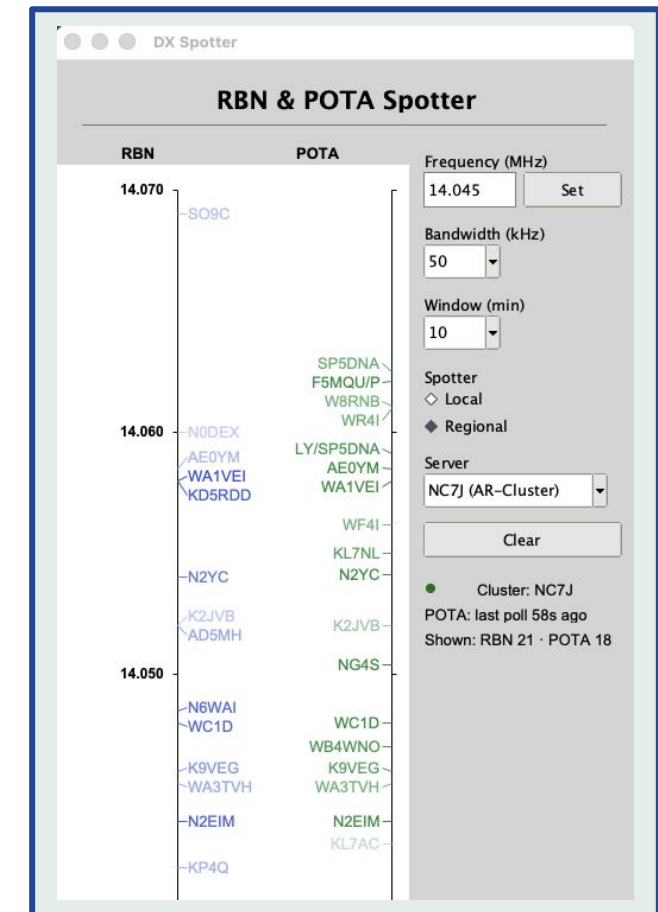
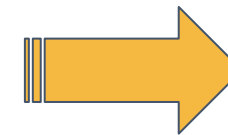
- POTA.APP shows maps, and spots for Hunting
 - Interface prioritizes mapping, not bandscope
- N1MM Windows app shows a bandscope, not POTA spots
- Field Spotter (Ian Renton) close to what I wanted



POTA.APP



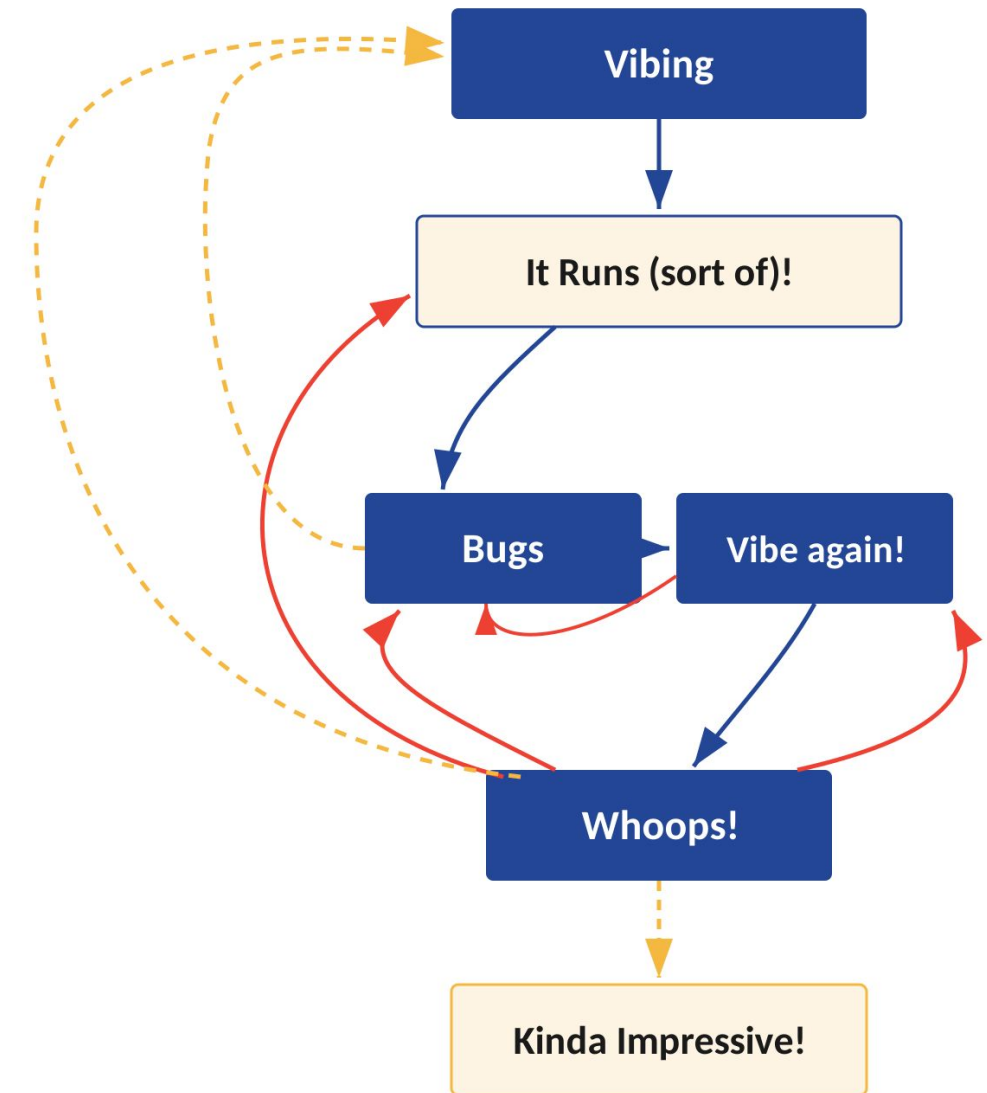
N1MM



SDD DX Spotter - Integrated

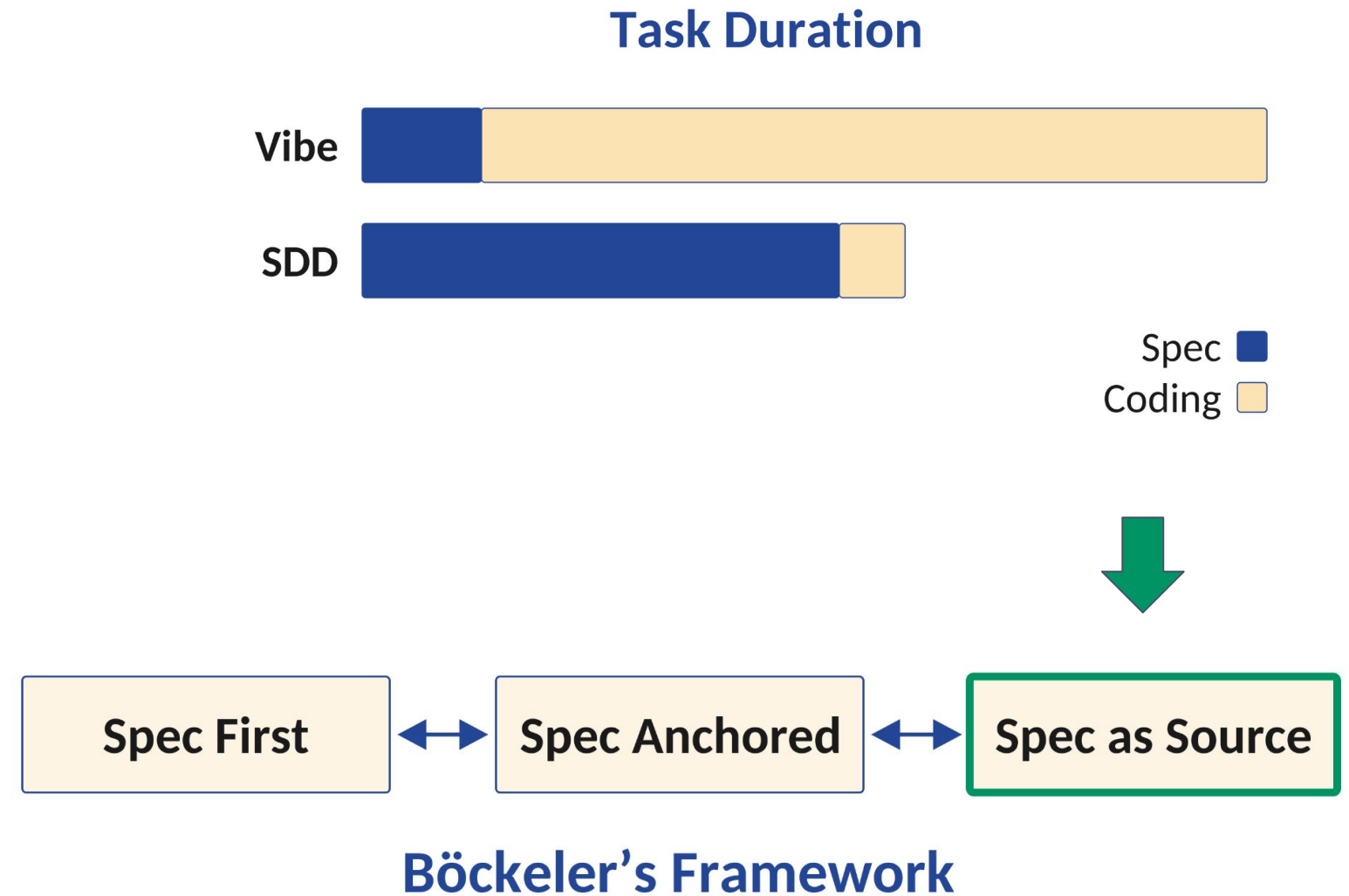
■ Vibe Coding, Can Be Amazing!

- Vague prompt: “create a spotting app for RBN & POTA” — **silent assumptions**
- Functionality: Display, clusters (CC Cluster, DX Spider, AR6?), UI, parameters — **all guessed, often wrong**
- Testing: Feature tests or just smoke test? — **results in brittle code**
- Now what if you want to come back 6 months from now to tweak it? — **where was that chat?**
- Vibe coding is amazing for what it can accomplish in the first 20% of time — **frequently frustrating**



■ A Better Alternative: SDD (Spec-Driven Development)

- SDD = (Spec + Build, Test, Update Spec)
- ‘vibing’ just was not working for large organizations
- Balance consistency and maintainability without impeding ultimate velocity
- Updating the spec more important than updating code
 - Spec replaces source code as deliverable
- Results in reduction in velocity
 - Better quality & maintainability



Vibe Prompt vs. SDD

	Vibe coding	Spec-driven
Prompt	“Create a spotting app for RBN & POTA”	idea.md + prompts that have the AI fill in the details
Details	✗ Clusters, UI - guessed, often wrong	✓ Written down, then confirmed or modified based on chats with AI
Testing	✗ Smoke test, if any	✓ Each step tested before commit
Speed	✓ Amazing first 20% ✗ Frustrating finish	✗ Slower start; more work up front
Later	✗ “Where is the chat?” ✗ “How can I change X?”	✓ Edit the spec; rebuild from GitHub ✓ Add features or change OS ✓ Repeatable and shareable

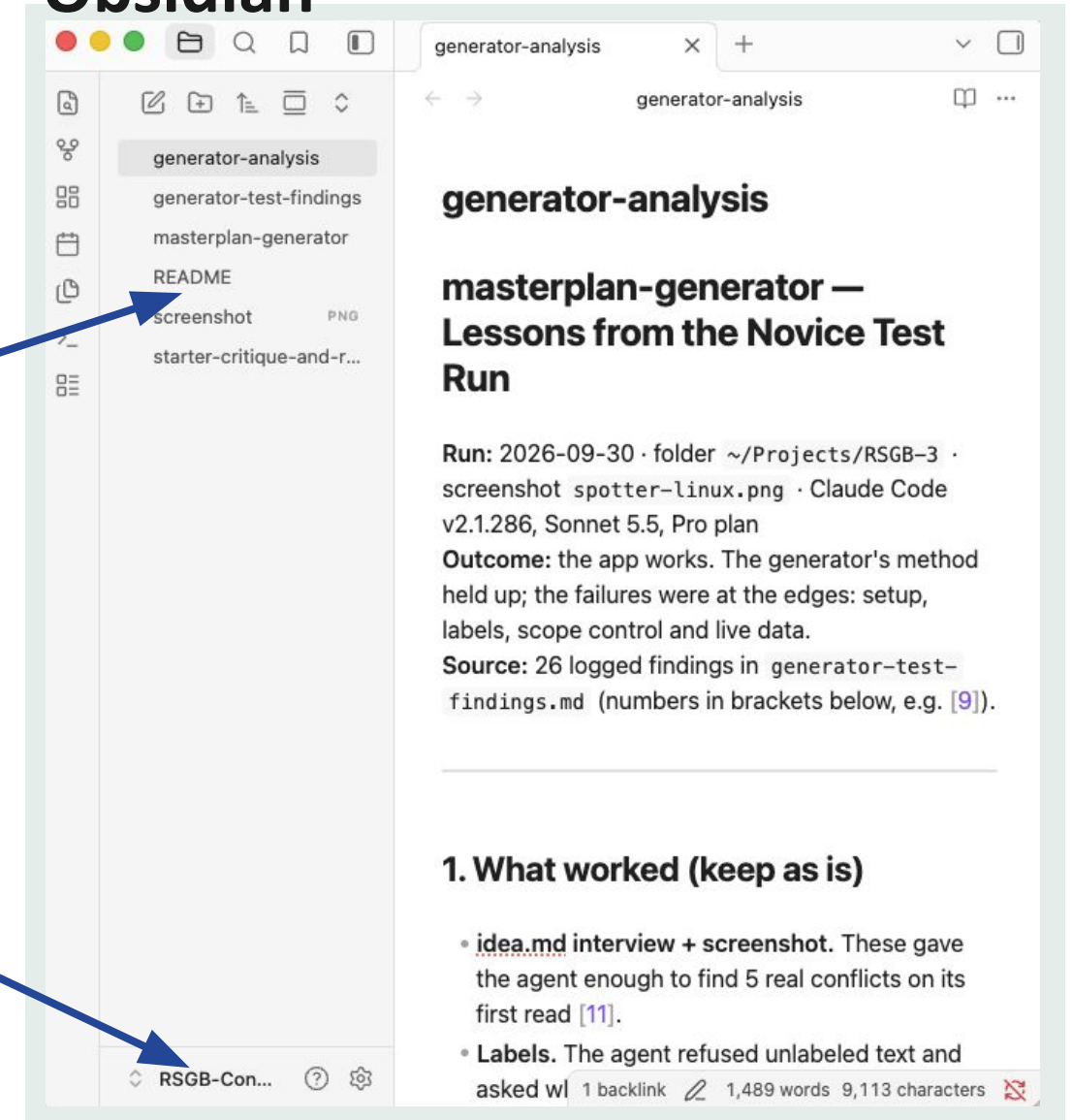
■ Preliminaries: Agentic AI CLI (Claude Code)

- Install Claude Code (\$20 Pro plan recommended)
 - Sonnet 5.5 model (low cost, good enough)
- Create project directory for initial files & generated code
- Install and setup
 - Setup GitHub (cloud), gh (local)
 - Authentication needed (can be hassle)
 - Google Drive, Chrome Connectors
- Install Obsidian as md file viewer
 - Pointed at project directory

Project
Files

Project
Directory

Obsidian



The Masterplan Example Prompt Outline

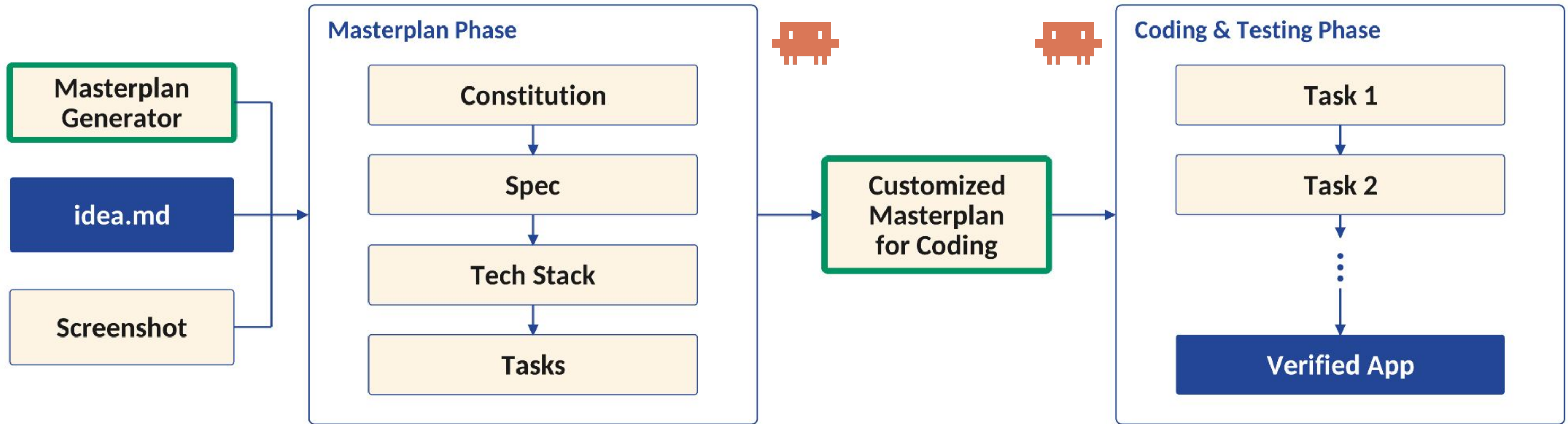
Section	Example prompt
1 Constitution how we work	“After each working step: run all tests, show me proof, commit, and push to GitHub.”
2 Spec what it does	“List what my app must do, and what it won’t do. Define inputs, data feeds.”
3 Tech built with	“Suggest a language and tools for my app; use Python unless my example or spec suggests better. Explain each.”
4 Tasks in what order	“For each step, say how we’ll test it and what result means it works.”

Use Generator + Your App Idea to Create Detailed Masterplan

Section	Typical Process
1 Constitution how we work	Generator prompts already sufficient, cut and paste into Claude Code and feed the rest of prompts for Spec, Tech, and Tasks
2 Spec what it does	Using app idea, and Spec questions from Masterplan Generator, ask Claude to customize Spec section
3 Tech built with	Use Masterplan sections 1 and 2 to ask Claude for recommended libraries and approaches to customize this section
4 Tasks in what order	Ask Claude to structure project into ~5 chunks: Setup, Connections, Core Logic, Main Features, UI (all from Generator)

■ Masterplan Generator + Idea + Screenshot

- The Masterplan Generator walks you through the creation of a masterplan spec tailored to your app
- Cut and paste generator prompts into Claude Code, which helps you create a detailed spec



Claude Code and Claude Desktop as Helper

dx_spotter/config.py DEFAULT_HOSTS and README.md:

Callsign	Host:port	Software	Status
NC7J	dxc.nc7j.com:7373	AR-Cluster v6.1.5123	Protocol captured live on 2026-09-04 (docs/protocol-notes.md)
WA9PIE-2	dxc.wa9pie.net:8000	DXSpider V1.57 build 686	Wired, verified live
W3LPL	w3lpl.net:7373	DXSpider	Wired
W4MYA	dxc.w4mya.us:7373	CC Cluster 3.397	Skimmer only, verified live

- **DXSpider:** WA9PIE-2 and W3LPL.
- **CC Cluster:** W4MYA only.
- **Login:** the app logs in as your own callsign, N6YU. The W4MYA prompt shows N6YU-0 de W4MYA.
- **NC7J:** it is AR-Cluster, not CC or DXSpider. I included it because it's the fourth default host.

Neither the README table nor DEFAULT_HOSTS says which of these four is tried first. If you don't set a preferred host, config.py uses the declared order, so NC7J goes first.

I cloned the repo into the session scratchpad, not ~/Projects/, and I didn't change anything in it.

* Churned for 13s · done 3:32 PM

* recap: You wanted to know which cluster servers and callsigns spotter-win2 uses for CC and DXSpider. Answered: DXSpider is WA9PIE-2 and W3LPL, CC Cluster is W4MYA. Next, tell me if you want anything changed or explored further (disable recaps in /config)

✓ Update installed · Restart to update

> Which callsigns did you host spots as?

▶ auto mode on (shift+tab to cycle) · ← for agents

R. / Masterplan-generat...

Reconsidering when and where to filter spot data. >

Approve 1, 2 and 4. Change 3: all four are real issues it found from live data, and it proposed them without editing the masterplan, which is what your Constitution asks for.

- 1, skimmer match: approve. W6YX-2-# is the real format. Check that the match stays exact on the base call and ignores upper/lower case, with the optional -<digits> and optional -# allowed only at the end.
- 2, POTA spot_id: approve. Without it, POTA spots would never fade, a bug you'd only notice after a few minutes of running. It's a good catch.
- 3, Local/Regional switching: change it. Keep spots from all Regional skimmers in the store, and apply Local at draw time.
 - Local is a subset of Regional, so nothing is lost.
 - Switching then redraws instantly instead of emptying the map.
 - It's the same pattern as pressure-test #4: filter at draw time, not on arrival.
 - Decision 6 changes to match.
- 4, late spots from the old band: approve.

Paste:

Approve 1, 2 and 4. Change 3: the store keeps spots from all Regional skimmers (Local is a subset); the Local/Regional choice applies at draw time, so switching redraws without emptying the map. Update decision 6 to say so. Add all four to masterplan.md, commit and push. Then go for Stage 4.

What can go wrong in Stage 4

- Screen Recording permission: the layout checks capture the app window. The first capture will fail, or come out blank, until you allow your terminal app under System Settings → Privacy & Security → Screen Recording. Then restart the terminal.

Unapproved network: the live PBN/POTA window will need it

Reply

+ Claude is AI and can make mistakes. Opus 5.5 Medium Manual

Code

Claude Desktop

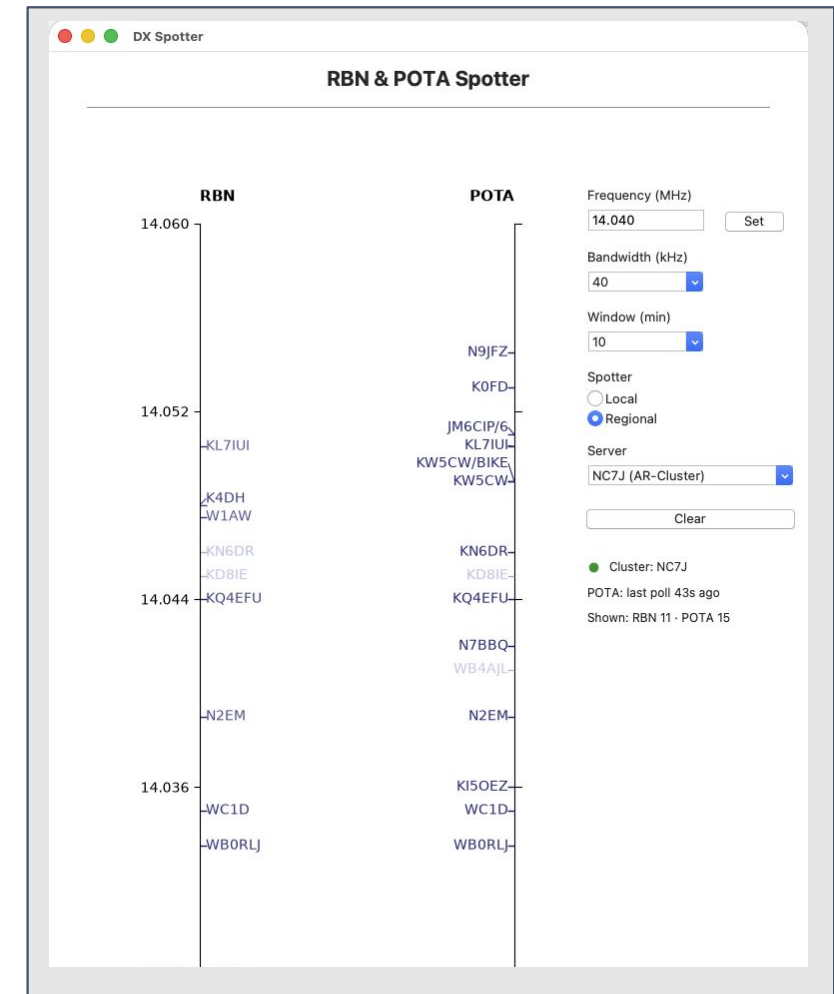
I don't understand output from Code!

Ask Claude Desktop for help

Paste answer back into Code

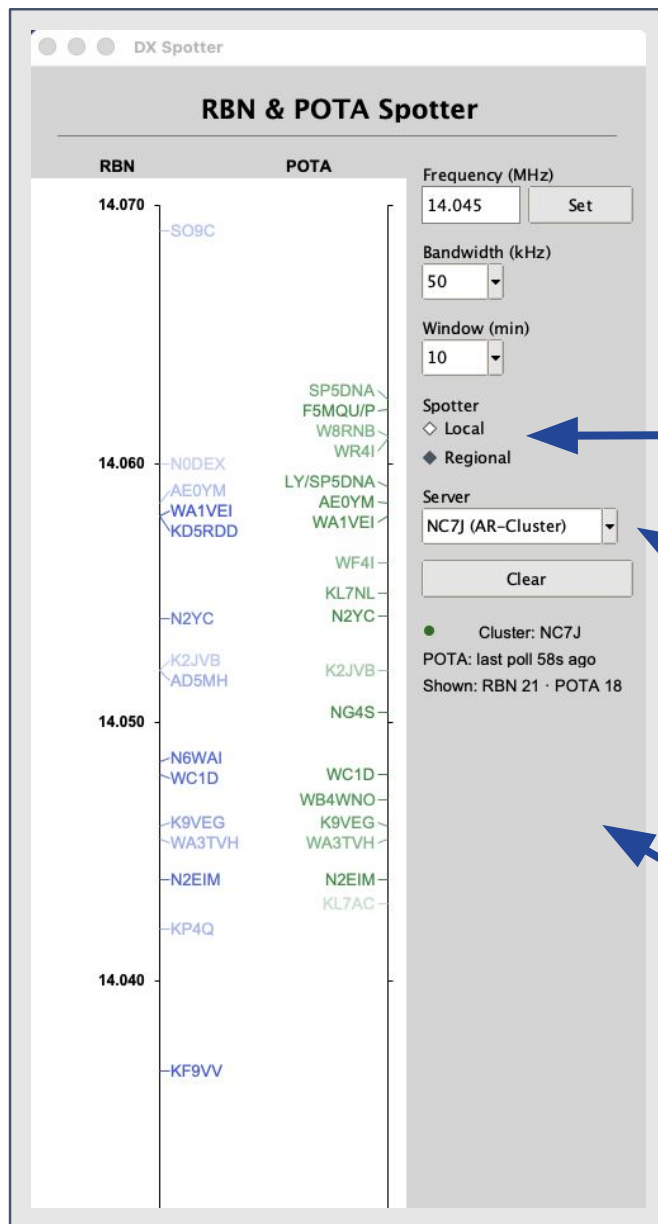
■ DX Spotter | The Working Result

- Bandmap: live RBN spots, adjustable window: working (vs N1MM)
- POTA.APP hunting view: working (vs POTA.APP)
- Frequency, bandscope window width, timeout menu
- Spotter selection for Local (within ~50 miles) and Regional (within ~250 miles)
- Cluster server menu (3 major server types supported)
- Status indication for RBN & POTA servers
- Out of scope today: DE (Spotter) specification, CAT control, mobile UI



■ Multiplatform Screenshots (Linux too)

Mac



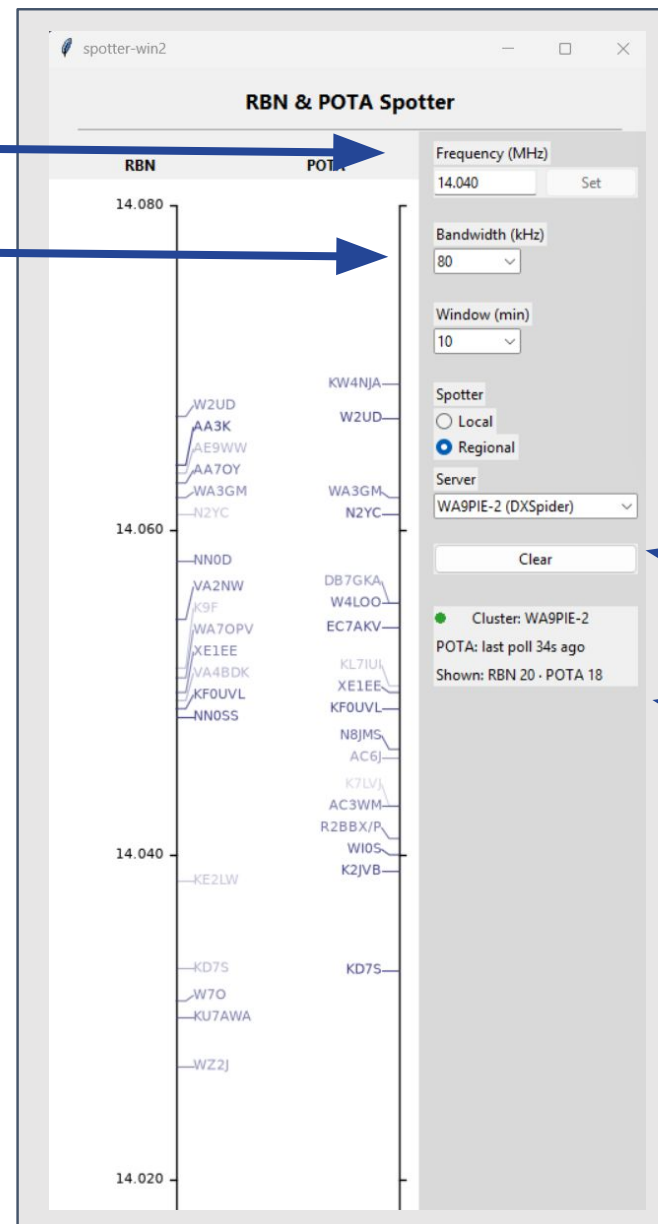
Center
Frequency
Bandwidth

Regional or
Local
Spotters

AR6
CC Cluster
DX Spider

Green POTA
Spots

Win



POTA Status

Spot Counts

Conclusion

Like a schematic, the spec is what you keep and share

Vibe coding gets you started. The Masterplan gets you finished.

1

Jot down the idea; then have agent build Masterplan...
...AND then have agent build code from it.

2

Build and test every step in task list, updating the
masterplan as you go, and GitHub remembering it.

3

Start small: Masterplan Generator with prompts, screenshot,
and your app idea.



Your first masterplan
starts here

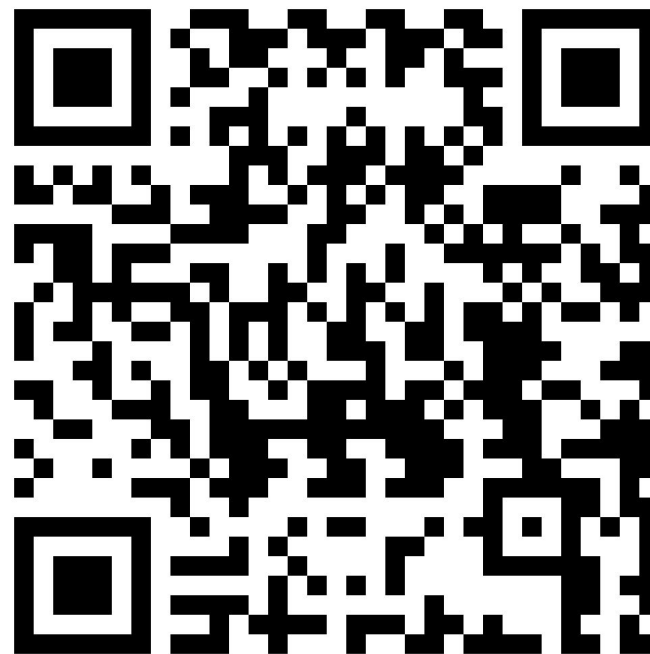
Appendix - GitHub Links



Talk repo

Slides, Example Masterplan,
Masterplan Generator,
References

github.com/jcarter-labs/rsgb-sdd-n6yu



Code Repo

DX Spotter App Repo

github.com/jcarter-labs/dx-spotter-app

Contact

John Carter

N6YU

john@n6yu.com

github.com/jcarter-labs

■ Appendix - References

- "Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants", Deepak Babu Piskala: <https://arxiv.org/pdf/2602.00180>
- Claude Code Tutorials (Anthropic short courses): <https://anthropic.skilljar.com/>
- Field Spotter (Ian Renton): <https://git.ianrenton.com/ian/field-spotter>
- Deeplearning (Practical short course on application of SDD): [Spec-Driven Development with Coding Agents - DeepLearning.AI](#)
- Slides, masterplans, references: github.com/jcarter-labs/rsgb-sdd-n6yu

■ Find out more...

John Carter, N6YU

john@n6yu.com

github.com/jcarter-labs

www.rsgb.org

